

Concrete Crack Detection (Semantic Segmentation)

This notebook demonstrates how to train a YOLO26 model for detecting concrete cracks, evaluate its performance, and deploy it as a Gradio web application.

Install Ultralytics

First, we need to install the `ultralytics` library, which provides the YOLO (You Only Look Once) model functionalities. The `-q` flag ensures a quiet installation.

```
!pip install -q ultralytics
```

Download Dataset

Next, we download the dataset for training the crack detection model from Kaggle Hub. The dataset used is "roads-and-bridges-cracks-yolov8-format". The `path` variable will store the location of the downloaded dataset.

```
import kagglehub
import os

path = kagglehub.dataset_download("rukiyeaydn/road-crack-dataset")
dataset_root = os.path.join(path, "road crack dataset semantic")
print("Dataset successfully downloaded to:", dataset_root)
```

Using Colab cache for faster access to the 'road-crack-dataset' dataset.
Dataset successfully downloaded to: /kaggle/input/road-crack-dataset/road crack dataset semantic

Directory Structure Inspection

This function recursively scans and displays a directory tree starting from a given target path, making it easy to inspect complex dataset structures (like Kaggle downloads).

Key Features:

- **Depth Control:** Limits folder traversal depth (`max_depth=3`) to prevent overwhelming output on deeply nested structures.
- **Output Truncation:** Displays up to **3 files per folder**; if a directory contains more, it displays a summary count (e.g., `... and N more files`).
- **Path Validation:** Checks whether the specified directory exists before attempting to traverse.

```
import os

def print_directory_tree(startpath, max_depth=3):
    print(f"Checking directory structure for: {startpath}\n")
    if not os.path.exists(startpath):
        print("Path does not exist!")
        return

    for root, dirs, files in os.walk(startpath):
        level = root.replace(startpath, '').count(os.sep)
        if level >= max_depth:
            continue
        indent = ' ' * 4 * (level)
        print(f"{indent}📁 {os.path.basename(root)}/")
        subindent = ' ' * 4 * (level + 1)

        # Show first 3 files in the folder to keep it clean
        for f in files[:3]:
            print(f"{subindent}📄 {f}")
```

```

if len(files) > 3:
    print(f"{subindent} ... and {len(files) - 3} more files")

# Scan the Kaggle download root directory
print_directory_tree(path)

```

Checking directory structure for: /kaggle/input/road-crack-dataset

```

road-crack-dataset/
├── README.md
├── road crack dataset semantic/
│   └── valid/
│       ├── road_1041_6_jpg.rf.3ecf39925e0966993a03c2edb50d59d6.jpg
│       ├── road_1043_6_jpg.rf.3bd8098099d3c9733cf3e745832aa773.jpg
│       ├── road_1041_2_jpg.rf.3676e8be6f216537f49e39f98098e794_mask.png
│       ├── ... and 171 more files
│   └── train/
│       ├── road_1001_2_jpg.rf.652c417741410e0fdf2e8547b6c68fe0.jpg
│       ├── road_1031_2_jpg.rf.a20f9dd0af2b55508b4861831be943be.jpg
│       ├── road_1041_1_jpg.rf.c7df8fbf42fd8958002d1b9e93993_mask.png
│       ├── ... and 693 more files

```

YOLO Dataset Preparation & Format Conversion

This step converts a read-only semantic segmentation dataset (binary masks) into a writable, native **YOLO Segmentation** format (`.txt` polygon coordinates).

Workflow Overview:

- Environment Setup:** Creates a writable working directory at `/content/crack_dataset/`.
- Mask-to-Polygon Extraction:**
 - Reads target mask images (`_mask.png`) in grayscale.
 - Extracts object contours using OpenCV (`cv2.findContours`).
 - Normalizes pixel coordinates relative to image width and height.
 - Saves polygon coordinates to `.txt` label files (`class_id x1 y1 x2 y2 ...`).
- Symlinking Images:** Creates soft links (`os.symlink`) to original images to save time and storage space.
- Configuration Generation:** Automatically builds the dataset metadata file (`data.yaml`) required for YOLO model training.

```

import os
import cv2
import yaml
from pathlib import Path

# Source dataset paths (Read-Only Kaggle Mount)
src_dataset_root = os.path.join(path, "road crack dataset semantic")

# Setup standard writable directories inside Colab's local space
writable_root = "/content/crack_dataset"
os.makedirs(writable_root, exist_ok=True)

def setup_yolo_split(split_name):
    # Establish local clean directories side-by-side as YOLO expects
    images_dst = os.path.join(writable_root, split_name, "images")
    labels_dst = os.path.join(writable_root, split_name, "labels")
    os.makedirs(images_dst, exist_ok=True)
    os.makedirs(labels_dst, exist_ok=True)

    # Source directory containing both images and masks mixed together
    split_src_dir = os.path.join(src_dataset_root, split_name)

    if not os.path.exists(split_src_dir):
        print(f"Directory {split_src_dir} does not exist!")
        return

    print(f"Processing split '{split_name}'...")

```

```

# Loop over the files in the flat directory
for file_name in os.listdir(split_src_dir):
    # Handle Mask Images
    if file_name.endswith('_mask.png'):
        mask_path = os.path.join(split_src_dir, file_name)
        mask = cv2.imread(mask_path, cv2.IMREAD_GRAYSCALE)
        if mask is None:
            continue

        contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

        # Match mask filename back to its corresponding image name
        img_base_name = file_name.replace('_mask.png', '.jpg')
        label_path = os.path.join(labels_dst, img_base_name.replace('.jpg', '.txt'))

        h, w = mask.shape
        with open(label_path, 'w') as f:
            for contour in contours:
                if len(contour) < 3:
                    continue
                contour = contour.reshape(-1, 2)
                normalized = [f"{x / w:.6f} {y / h:.6f}" for x, y in contour]
                f.write(f"0 {' '.join(normalized)}\n")

    # Handle Normal Images (skip masks)
    elif file_name.lower().endswith((' .jpg', '.jpeg', '.png')):
        src_img_path = os.path.join(split_src_dir, file_name)
        dst_img_path = os.path.join(images_dst, file_name)
        if not os.path.exists(dst_img_path):
            os.symlink(src_img_path, dst_img_path)

# Restructure train and valid splits smoothly
setup_yolo_split("train")
setup_yolo_split("valid")

# Write a pristine, native YOLO configuration file targeting our new folders
yaml_path = os.path.join(writable_root, "data.yaml")
data_yaml_content = {
    'path': writable_root,
    'train': 'train/images',
    'val': 'valid/images',
    'names': {
        0: 'Concrete Crack',
        1: 'Construction Joints',
        2: 'Drilling'
    }
}

with open(yaml_path, 'w') as f:
    yaml.dump(data_yaml_content, f, default_flow_style=False)

print(f"\nRestructuring complete! Native configuration ready at: {yaml_path}")

```

```

Processing split 'train'...
Processing split 'valid'...

```

```

Restructuring complete! Native configuration ready at: /content/crack_dataset/data.yaml

```

▾ Train YOLO26 Nano Segment Model

We initialize a pre-trained `yolo26n-seg.pt` (nano version) model and then train it using our downloaded dataset. The training process is configured with 50 epochs and an image size of 640x640 pixels.

```
from ultralytics import YOLO

# load pre-trained model
model = YOLO("yolo26n-seg.pt")

# train the model
model.train(
    data=yaml_path,
    epochs=50,
    imgsz=640,
    task="segment",
)
```

New <https://pypi.org/project/ultralytics/8.4.102> available 🤗 Update with 'pip install -U ultralytics'
Ultralytics 8.4.101 🚀 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=randaugment, batch=16, bgr=0.0, box=7.5, c
Overriding model.yaml nc=80 with nc=3

	from	n	params	module	arguments	
0		-1	1	464	ultralytics.nn.modules.conv.Conv	[3, 16, 3, 2]
1		-1	1	4672	ultralytics.nn.modules.conv.Conv	[16, 32, 3, 2]
2		-1	1	6640	ultralytics.nn.modules.block.C3k2	[32, 64, 1, False, 0.25]
3		-1	1	36992	ultralytics.nn.modules.conv.Conv	[64, 64, 3, 2]
4		-1	1	26080	ultralytics.nn.modules.block.C3k2	[64, 128, 1, False, 0.25]
5		-1	1	147712	ultralytics.nn.modules.conv.Conv	[128, 128, 3, 2]
6		-1	1	87040	ultralytics.nn.modules.block.C3k2	[128, 128, 1, True]
7		-1	1	295424	ultralytics.nn.modules.conv.Conv	[128, 256, 3, 2]
8		-1	1	346112	ultralytics.nn.modules.block.C3k2	[256, 256, 1, True]
9		-1	1	164608	ultralytics.nn.modules.block.SPPF	[256, 256, 5, 3, True]
10		-1	1	249728	ultralytics.nn.modules.block.C2PSA	[256, 256, 1]
11		-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
12	[-1, 6]	1	0	ultralytics.nn.modules.conv.Concat	[1]	
13		-1	1	119808	ultralytics.nn.modules.block.C3k2	[384, 128, 1, True]
14		-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
15	[-1, 4]	1	0	ultralytics.nn.modules.conv.Concat	[1]	
16		-1	1	34304	ultralytics.nn.modules.block.C3k2	[256, 64, 1, True]
17		-1	1	36992	ultralytics.nn.modules.conv.Conv	[64, 64, 3, 2]
18	[-1, 13]	1	0	ultralytics.nn.modules.conv.Concat	[1]	
19		-1	1	95232	ultralytics.nn.modules.block.C3k2	[192, 128, 1, True]
20		-1	1	147712	ultralytics.nn.modules.conv.Conv	[128, 128, 3, 2]
21	[-1, 10]	1	0	ultralytics.nn.modules.conv.Concat	[1]	
22		-1	1	463104	ultralytics.nn.modules.block.C3k2	[384, 256, 1, True, 0.5, True]
23	[16, 19, 22]	1	791341	ultralytics.nn.modules.head.Segment26	[3, 32, 64, 1, True, [64, 128, 256]]	

YOLO26n-seg summary: 309 layers, 3,053,965 parameters, 3,053,965 gradients, 10.2 GFLOPs

Transferred 740/844 items from pretrained weights
AMP: running Automatic Mixed Precision (AMP) checks...
AMP: checks passed ✓
train: Fast image access ✓ (ping: 0.0±0.0 ms, read: 209.5±70.3 MB/s, size: 117.2 KB)
train: Scanning /content/crack_dataset/train/labels... 348 images, 10 backgrounds, 0 corrupt: 100% ————— 348/348 493.1it/s
train: New cache created: /content/crack_dataset/train/labels.cache
augmentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average')
WARNING ⚠ val: Slow image access detected (ping: 0.0±0.0 ms, read: 20.2±6.5 MB/s, size: 94.3 KB). Use local storage instead of Google Drive
val: Scanning /content/crack_dataset/valid/labels... 87 images, 0 backgrounds, 0 corrupt: 100% ————— 87/87 135.6it/s 0.0it/s
val: New cache created: /content/crack_dataset/valid/labels.cache
optimizer: 'optimizer=auto' found, ignoring 'lr=0.01' and 'momentum=0.937' and determining best 'optimizer', 'lr' and 'momentum' from **optimizer:** AdamW(lr=0.001429, momentum=0.9) with parameter groups 134 weight(decay=0.0), 154 weight(decay=0.0005), 154 bias(decay=0.0)
Plotting labels to /content/runs/segment/train-13/labels.jpg...
Image sizes 640 train, 640 val
Using 2 dataloader workers
Logging results to /content/runs/segment/train-13
Starting training for 50 epochs...

Epoch	GPU_mem	box_loss	seg_loss	cls_loss	dfl_loss	sem_loss	Instances	Size	640: 100%	22/22 1
1/50	3.31G	2.064	4.254	5.036	0.02138	6.998	92			
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95)	Mask(P	R	mAP50 mAP
	all	87	336	0.0082	0.577	0.0385	0.0136	0.00253	0.179	0.00597 0.0
Epoch	GPU_mem	box_loss	seg_loss	cls_loss	dfl_loss	sem_loss	Instances	Size	640: 100%	22/22 1
2/50	3.91G	1.67	2.526	4.518	0.01546	5.157	86			

Export & Download Trained Model

After training, we export the `best.pt` weights of the model to the ONNX (Open Neural Network Exchange) format. ONNX is an open standard designed to represent machine learning models, which allows for easier deployment across different platforms and hardware. This cell provides functionality to download the exported `best.onnx` model and the pytorch model file directly to local machine. This allows us to use the trained model outside of this Colab environment.

```
from google.colab import files

# Export the trained weights to ONNX
model.export(format="onnx")

# Define the path to the best.pt file
pt_model_path = "/content/runs/segment/train-13/weights/best.pt"

# Define the path to the best.onnx file
onnx_model_path = "/content/runs/segment/train-13/weights/best.onnx"

def download_model_file(file_path, file_type):
    """Helper function to download a specified model file."""
    try:
        files.download(file_path)
        print(f"Downloading {file_path} ({file_type}).")
    except Exception as e:
        print(f"Error downloading the {file_type} model from {file_path}: {e}")
        print(f>Please ensure the {file_type} file path is correct and the file exists.")

# Download the .pt file
download_model_file(pt_model_path, ".pt")

# Download the .onnx file
download_model_file(onnx_model_path, ".onnx")
```

Ultralytics 8.4.101 🚀 Python-3.12.13 torch-2.11.0+cu128 CPU (Intel Xeon CPU @ 2.00GHz)

💡 ProTip: Export to OpenVINO format for best performance on Intel hardware. Learn more at <https://docs.ultralytics.com/integrations/openvino>
YOLO26n-seg summary (fused): 139 layers, 2,689,469 parameters, 0 gradients, 9.0 GFLOPs

PyTorch: starting from '/content/runs/segment/train-13/weights/best.pt' with input shape (1, 3, 640, 640) BCHW and output shape(1, 1, 1, 1) B1C1H1W1
requirements: Ultralytics requirements ['onnx>=1.12.0,<2.0.0', 'onnxruntime', 'onnxslim>=0.1.82'] not found, attempting AutoUpdate
Using Python 3.12.13 environment at: /usr

Resolved 12 packages in 408ms

Prepared 4 packages in 2.21s

Installed 4 packages in 406ms

+ colorama==0.4.6

+ onnx==1.22.0

+ onnxruntime==1.27.0

+ onnxslim==0.1.94

requirements: AutoUpdate success ✅ 3.7s

WARNING ⚠️ **requirements:** Restart runtime or rerun command for updates to take effect

ONNX: starting export with onnx 1.22.0 opset 20...

/usr/local/lib/python3.12/dist-packages/torch/onnx/_internal/torchscript_exporter/symbolic_opset11.py:954: UserWarning: Exporting return opset9.index(g, self, index)

ONNX: slimming with onnxslim 0.1.94...

ONNX: export success ✅ 5.9s, saved as '/content/runs/segment/train-13/weights/best.onnx' (10.6 MB)

Export complete (6.5s)

Results saved to /content/runs/segment/train-13/weights/best.onnx

Predict: yolo predict task=segment model=/content/runs/segment/train-13/weights/best.onnx imgsz=640

Validate: yolo val task=segment model=/content/runs/segment/train-13/weights/best.onnx imgsz=640 data=/content/crack_data

Visualize: <https://netron.app>

Downloading /content/runs/segment/train-13/weights/best.pt (.pt) to your local machine.

Downloading /content/runs/segment/train-13/weights/best.onnx (.onnx) to your local machine.

✦ Visualize Training Results

During training, Ultralytics generates various plots to visualize the model's performance and training progress. This section displays key evaluation plots like `results.png` (showing loss and mAP over epochs), `confusion_matrix.png`, and `F1_curve.png`.

```
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import os

# Path to the results directory (based on your logs: train-3)
results_dir = '/content/runs/detect/train-3'

# List of evaluation plots to view
plots = ['results.png', 'confusion_matrix.png', 'F1_curve.png']

for plot in plots:
    plot_path = os.path.join(results_dir, plot)
    if os.path.exists(plot_path):
        plt.figure(figsize=(10, 8))
        img = mpimg.imread(plot_path)
        plt.imshow(img)
        plt.axis('off')
        plt.title(plot)
        plt.show()
```

✓ Validate Model Performance

This cell performs a final validation of the trained model using the `best.pt` weights and the validation split of the dataset. It prints important metrics such as mAP50 (mean Average Precision at an IoU threshold of 0.5) and mAP50-95 (mean Average Precision across IoU thresholds from 0.5 to 0.95), which are standard metrics for object detection tasks.

```
# Validate the model using the best weights generated
metrics = model.val(data=yaml_path)
```

```
# Print key metrics cleanly
print(f"Box mAP50: {metrics.box.map50:.4f}")
print(f"Mask mAP50: {metrics.seg.map50:.4f}")
```

```
Ultralytics 8.4.101 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
val: Fast image access (ping: 0.0±0.0 ms, read: 150.1±83.6 MB/s, size: 92.3 KB)
val: Scanning /content/crack_dataset/valid/labels.cache... 87 images, 0 backgrounds, 0 corrupt: 100% 87/87 26.1Mit/
```

	Class	Images	Instances	Box(P)	R	mAP50	mAP50-95)	Mask(P)	R	mAP50	mAP50
	all	87	336	0.617	0.384	0.433	0.229	0.361	0.229	0.184	0.
	Concrete Crack	87	336	0.617	0.384	0.433	0.229	0.361	0.229	0.184	0.

```
Speed: 9.4ms preprocess, 10.0ms inference, 0.0ms loss, 5.3ms postprocess per image
Results saved to /content/runs/segment/val-3
Box mAP50: 0.4326
Mask mAP50: 0.1838
```

✓ Deploy with Gradio UI

Finally, we create a simple web interface using `Gradio` to interact with our trained crack detection model. You can upload an image, adjust the confidence threshold, and the application will display the image with detected cracks and a status message.

```
import gradio as gr
from ultralytics import YOLO
import cv2

# 1. Load your freshly trained custom segmentation model weights
model_path = "/content/runs/segment/train-13/weights/best.pt"
model = YOLO(model_path)

def predict_cracks(image):
    if image is None:
        return None, "Please upload an image."

    results = model(image)
```

```

result = results[0]

if result.masks is not None and len(result.masks) > 0:
    # FIXED: boxes=False hides the rectangular bounding box contours
    # FIXED: labels=False removes the 'Concrete Crack 0.XX' text overlays
    annotated_img = result.plot(boxes=False, labels=False)

    crack_count = len(result.masks)
    status_text = f"Success: Detected {crack_count} concrete crack segments!"

    annotated_img = cv2.cvtColor(annotated_img, cv2.COLOR_BGR2RGB)
    return annotated_img, status_text
else:
    return image, "Status: No concrete cracks detected in this sample."

# 2. Build the Segment-specific Gradio Interface
interface = gr.Interface(
    fn=predict_cracks,
    inputs=gr.Image(type="numpy", label="Upload Concrete Image"),
    outputs=[
        gr.Image(type="numpy", label="Semantic Segmentation Output"),
        gr.Textbox(label="Detection Status")
    ],
    title="Concrete Crack Detection UI (Semantic Segmentation)",
    description="Upload a concrete surface photo. The custom YOLO model will overlay precise pixel",
    theme="interactive"
)

# Launch the app with a public share link
interface.launch(share=True)

```

/usr/local/lib/python3.12/dist-packages/gradio/interface.py:171: UserWarning: The parameters have been moved from the Blocks constructor to the GradioInterface constructor.
 super().__init__(
 /usr/local/lib/python3.12/dist-packages/gradio/utils.py:583: UserWarning: Cannot load interactive. Caught Exception: Client error
 For more information check: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status/404>

Sorry, we can't find the page you are looking for.
 warnings.warn(f"Cannot load {theme}. Caught Exception: {str(e)}")
 Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
 * Running on public URL: <https://586fdffa269ed17236.gradio.live>

This share link is temporary and will last for up to 1 week (best effort). For free permanent hosting and GPU upgrades, run `gra



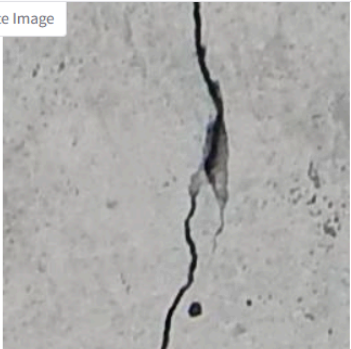
No interface is running right now

Model Output 1

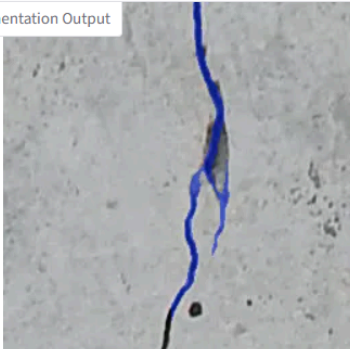
Concrete Crack Detection UI (Semantic Segmentation)

Upload a concrete surface photo. The custom YOLO model will overlay precise pixel masks over any detected cracks or defects.

Upload Concrete Image



Semantic Segmentation Output



Detection Status

Success: Detected 1 concrete crack segments!

Model Output 2

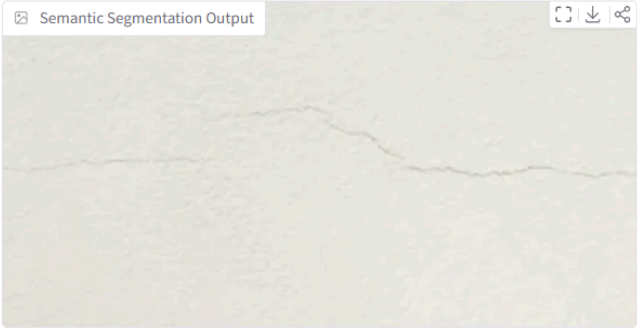
Concrete Crack Detection UI (Semantic Segmentation)

Upload a concrete surface photo. The custom YOLO model will overlay precise pixel masks over any detected cracks or defects.

Upload Concrete Image



Semantic Segmentation Output



Detection Status

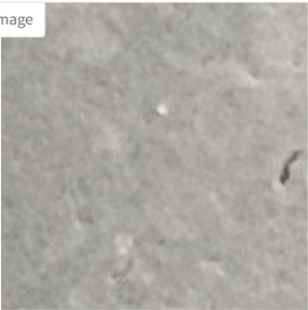
Status: No concrete cracks detected in this sample.

Model Output 3

Concrete Crack Detection UI (Semantic Segmentation)

Upload a concrete surface photo. The custom YOLO model will overlay precise pixel masks over any detected cracks or defects.

Upload Concrete Image



↶

↷

✕

Semantic Segmentation Output



↶

↷

⌵

⌵

⌵

Clear

Submit

Detection Status

Status: No concrete cracks detected in this sample.

Start coding or [generate](#) with AI.